Among the custom methods you defined in `BookRepository`, `deleteByIsbn()` is a good candidate for being transactional, since it modifies the database state. You can ensure it runs in a transaction by applying the `@Transactional` annotation.

Listing 5.15 Defining transactional operations

```
package com.polarbookshop.catalogservice.domain;

import java.util.Optional;
import org.springframework.data.jdbc.repository.query.Modifying;
import org.springframework.data.jdbc.repository.query.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.transaction.annotation.Transactional;

public interface BookRepository extends CrudRepository<Book,Long> {

  Optional<Book> findByIsbn(String isbn);
  boolean existsByIsbn(String isbn);

  @Modifying
  @Transactional                                    ❶
  @Query("delete from Book where isbn = :isbn")
  void deleteByIsbn(String isbn);
}
```

❶ Identifies the method to be executed in a transaction

NOTE For more information about the declarative transaction management provided by Spring Framework, you can refer to the official documentation (https://spring.io/projects/spring-framework).

Great job! You successfully added data persistence capabilities to the Catalog Service application. Let's verify that it works correctly. First of all, make sure the PostgreSQL container is still running. If not, run it as described at the beginning of this chapter. Then start the application (`./gradlew bootRun`), send HTTP requests to each REST endpoint, and ensure it works as expected. When you're done, remove the database container (`docker rm -fv polar-postgres`) and stop the application (Ctrl-C).